

Regular expression - RegEx

Регулярни изрази – както показва името им, регулират текст въз основа на шаблон. Те улесняват търсенето, валидирането и като цяло манипулирането на текст.

Основни класове

- **Regex** - основен клас за работа с регулярни изрази
- **Match** - резултат от единично съвпадение
- **MatchCollection** - колекция от съвпадения
- **Group** - група от съвпадения
- **Capture** - единично съвпадение

Щом са класове – могат да създават нови обекти чрез **new** по зададен шаблон.

Често срещани метасимволи

- `.` - all - всеки символ (без нов ред)
- `\d` – digit - цифра [0-9]
- `\D` – non-digit - не-цифра – Главните букви дават смисъл на НЕ
- `\w` - word - буквено-цифров символ [a-zA-Z0-9_]
- `\W` - не-буквено-цифров символ
- `\s` – (white)space – празен символ – интервал, табулация, нов ред
- `\S` - не-whitespace символ
- `^` - начало на низ/ред
- `$` - край на низ/ред - глобален разделител в SQL
- `\b` - border - граница на дума
- `|` - или (алтернатива) – има смисъл на ИЛИ, но е 1.5 пъти по-бавно
- `[]` - набор от символи
- `[^]` – НЕ - отрицание на набор

Лявата наклонена черта `\` е escape-последователност – показва команда, която пренебрегва стоящият след нея символ, т.е. придава му специално (а не дословно) значение като `\n` = new line за нов ред, `\"` – отпечатва двойни кавички...

Освен двойните И `&&` и ИЛИ `||`, които сме свикнали да използваме, има и единични `&` и `|`, при които, обаче, се губи повече ресурс, защото:

`&&` - и двете условия едновременно трябва да са верни – ако първото е грешно, не проверява второто. Единичното `&` го проверява, с което губи ресурси.

`||` - ако първото условие е вярно, целият израз се приема за верен и дори не се проверява втория. Единичното `|` проверява и него, с което също губи ресурси.

Квантификатори

- * - 0 или повече
- + - 1 или повече
- ? - 0 или 1
- {n} - точно n пъти
- {n,} - поне n пъти
- {n,m} - от n до m пъти

Шаблони

```
string emailPattern = @"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$";

string phonePattern = @"^\+?[0-9\s\-\(\)]{10,}$";

// URL адрес
string urlPattern = @"^(https?:\/\/)?([\da-z\.-]+)\.([a-z\.]{2,6})([\/\w\.-]*)*\/?$";

// Дата (dd/mm/yyyy)
string datePattern = @"^(0[1-9]|[12][0-9]|3[01])\/(0[1-9]|1[0-2])\/\d{4}$";

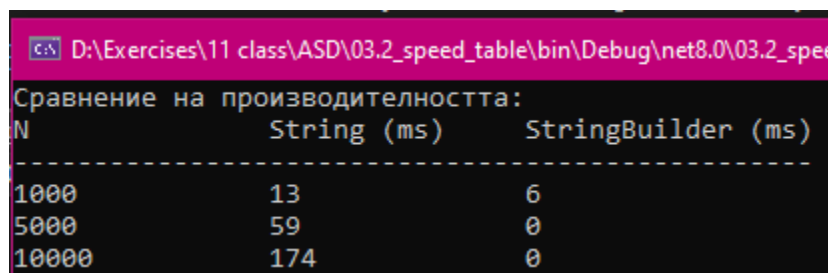
// Парола (минимум 8 знака, поне една буква и цифра)
string passwordPattern = @"^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$";
```

Задача 2: Сравнение на производителността (String vs StringBuilder)

Условие:

Използвайки като идея кода от предходния таймер, направете подобен експеримент, който сравнява скоростта на работа на string и StringBuilder:

1. Създайте два метода - първият използва string конкатенация, другият StringBuilder
2. Всеки метод трябва да построи низ, съдържащ числата от 1 до N, разделени със запетаи
3. Измерете времето за изпълнение на всеки метод за различни стойности на N (1000, 10000, 50000)
4. Създайте таблица с резултатите
5. Направете изводи кога е по-добре да се използва всеки от подходите



Сравнение на производителността:

N	String (ms)	StringBuilder (ms)
1000	13	6
5000	59	0
10000	174	0